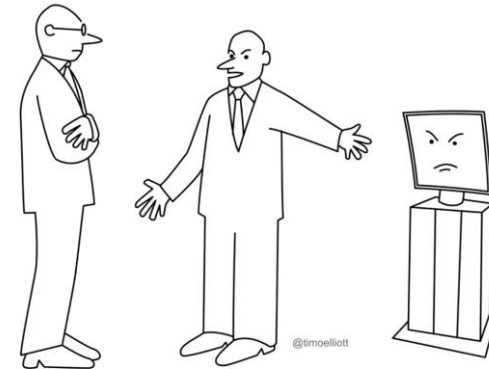


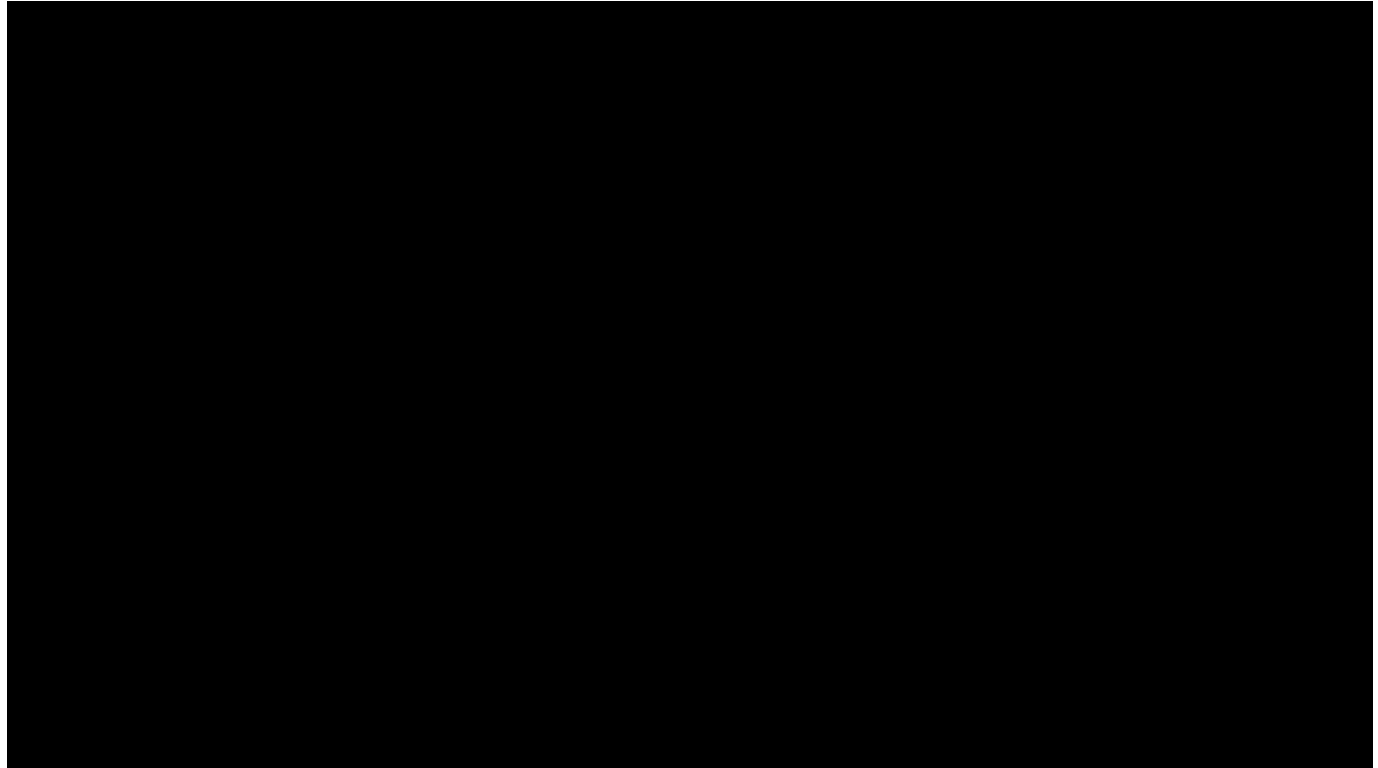
Artificial Intelligence (AI) & Machine Learning (ML) in Cyber Security

HSLU I - Modul SOC.F2601



*His decisions aren't any better than yours
— but they're WAY faster..*

Intro



Programm

- ML & AI – modernes Buzzword Bingo?
- Begriffe / Übersicht
- Abgrenzung
- Anwendung in CyberSecurity
- KI-Grundlagen
- ML-Grundlagen
- Datenbasis & Datenvorbereitung
- Anhang
 - Neuronale Netze
 - Quellen

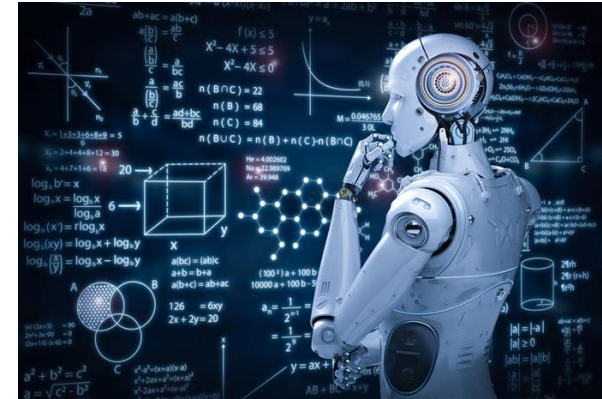


Bild-Quelle:

<https://www.securityinfowatch.com/cybersecurity/article/21114214/a-brief-history-of-machine-learning-in-cybersecurity>

Mehr als Buzzwords?

- Heutzutage lässt sich praktisch kein Produkt für den CyberSecurity Bereich ohne diese Schlagworte verkaufen
- Das lässt neugierig werden:
 - was steckt dahinter?
 - und funktioniert es?

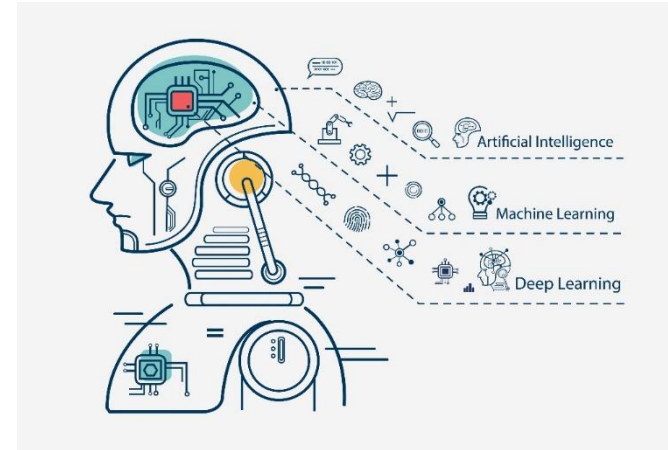


Bild-Quelle: <https://towardsdatascience.com/is-the-future-of-cyber-security-in-the-hands-of-artificial-intelligence-ai-1-2b4bd8384329>

Produkte ...

DARKTRACE Technology Products Industry

World leaders in Autonomous Cyber AI

The Darktrace Immune System is the world's leading autonomous cyber defense platform. Its award-winning **Cyber AI** protects your workforce and data from sophisticated attackers, by detecting, investigating and responding to cyber-threats in real time — wherever they strike.

CloudGuard for Cloud Intelligence and Threat Hunting

CloudGuard Cloud Intelligence and Threat Hunting provides native threat security forensics through visualization, giving real-time context of activity across your multi-cloud environment.

Platform / AI Detection and Response

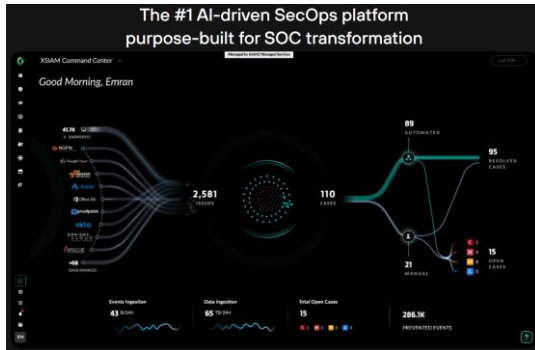
Stop AI breaches

From the pioneer of EDR, MDR, and CDR, secure the AI attack surface with AIDR.

[Download eBook →](#) [Schedule a demo →](#) [Watch video](#)

EnterpriseSearchChat
WebsiteSupportChatbot
Notion AI
AI Studio
ChatGPT
Claude
Gemini
Mistral
OpenAI
Sonar
DeepSeek

18 Detections from 54 Events
Malicious Prompt



VECTRA Produkte und Services Lösungen Ressourcen Partner Über uns [Lernen Sie mehr](#)

Die Cognito-Plattform:

KI-gestützte Bedrohungserkennung und Response in der Cloud

Korrelation mit Sicherheitsdaten angereicherter Metadaten mit anderen Quellen

Schlüssiges Threat Hunting und Untersuchungen von Vorfällen

Automatische Erkennung verborgener Angriffe in Echtzeit

Füllen Sie Ihre Data Lakes und SDBs mit Netzwerk-Metadaten im Zeil-Format, die mit Sicherheitsdaten angereichert wurden.

Die Untersuchungs-Werkbank ist für angereicherte Metadaten optimiert und ermöglicht umfangreiche Suchen in Sekundenbruchteilen.

Laufende Angriffe auf kompromittierte Host-Geräte können erkannt, priorisiert und korrigiert werden.

BLUEHEXAGON Products Solutions

Agentless Cloud-Native AI Security

- Real-time Threat Defense
- Actionable Visibility
- Continuous Compliance

With Cloud Runtime and Hybrid Network Deployment

[LEARN MORE](#) [TEST DRIVE](#)

AVAILABLE NOW

Blue Hexagon Agentless AI-Security for AWS

[Read Our Latest Update](#)

aber ...



Utah: Prüfer finden keine KI bei KI-Überwachungsfirma Banjo

Der US-Bundesstaat stellte eine von ihm beschaffte Software für Predictive Policing des "KI-Startups" Banjo auf den Prüfstand und erlebt eine Überraschung.

08.04.2021 19:31 Uhr  27

Quelle: <https://www.heise.de/news/Utah-Pruefer-finden-keine-KI-bei-KI-Ueberwachungsfirma-Banjo-6009371.html>

Begriffsdefinitionen (1)

- **KI (Künstliche Intelligenz)** - ein weit gefasster Begriff. Eine Wissenschaft welche die Dinge intelligent macht oder mit anderen Worten: menschliche Aufgaben die von Maschinen ausgeführt werden (z.B. Visuelle Erkennung, NLP, etc.).

Der Hauptpunkt ist aber, dass KI nicht genau maschinelles Lernen oder intelligente Dinge ist. Es kann ein klassisches Programm sein das in einem Reinigungsroboter installiert ist, z. B. für Kantenerkennung.

→ Grob gesagt ist KI ein Ding, das irgendwie menschliche Aufgaben ausführt.

Begriffsdefinitionen (2)

- **ML (Machine Learning)** - ein Ansatz der KI (nur einer von vielen Ansätzen) welcher ein System verwendet das in der Lage, ist aus Erfahrungen zu lernen. Es ist nicht nur für KI-Ziele gedacht (z. B. kopieren von menschlichem Verhalten) sondern kann auch den Aufwand und/oder die Zeit für sowohl einfache als auch schwierige Aufgaben wie Gesichts- bzw. Objekterkennung reduzieren. Mit anderen Worten: **ML ist ein System, das Muster anhand von Beispielen erkennen kann, anstatt sie zu programmieren.**

Wenn ein System ständig lernt, Entscheidungen auf Basis von Daten und nicht von Algorithmen trifft und sein Verhalten ändert, handelt es sich um ***Machine Learning***.

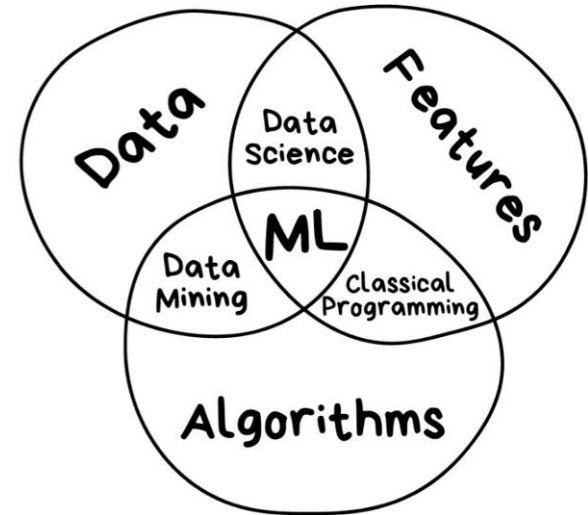
Begriffsdefinitionen (3)

- **DL (Deep Learning)** - eine Reihe von Techniken zur Implementierung von maschinellem Lernen um die Muster von Mustern zu erkennen - wie z.B. die Bilderkennung. Die Systeme erkennen in erster Linie Objektkanten, eine Struktur, einen Objekttyp und dann ein Objekt selbst. Es gibt andere Algorithmen die verbessert wurden um Muster von Mustern zu lernen, wie z. B. Deep Q Learning in Reinforcement Task (Verstärkungsaufgaben).
- **Neuronale Netze** sind nun als *Teil des überwachten Lernens* wiederum das derzeit relevanteste Teilgebiet des maschinellen Lernens. Sie in der Lage, grosse Mengen an unstrukturierten Daten besonders gut auszuwerten und z.B. Muster in ihnen zu finden. Zu unstrukturierten Daten gehören Bilder, Videos oder Töne, also all jene Daten, wir in unserem Alltag in grosser Menge produzieren.

Übersicht und Einordnung

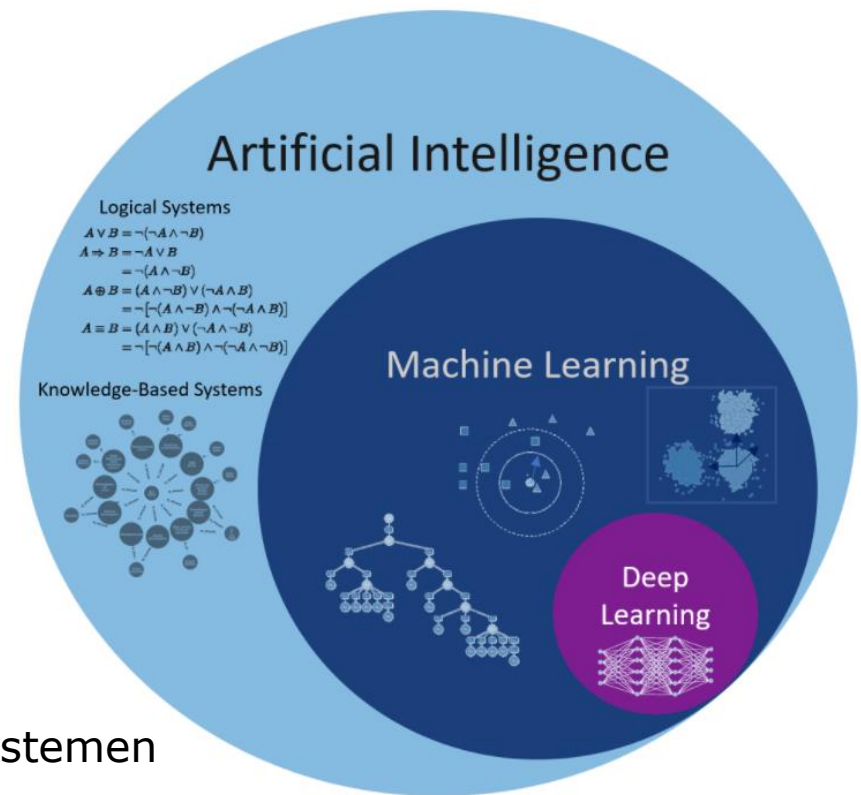
Die Definitionen zeigen, dass sich der Bereich CyberSecurity hauptsächlich auf maschinelles Lernen (nicht auf KI) bezieht. Und ein grosser Teil der Aufgaben nicht menschenbezogen ist.

Maschinelles Lernen bedeutet, bestimmte Aufgaben mit Hilfe eines Ansatzes und bestimmter Methoden zu lösen, auf der Grundlage von Daten die man besitzt.



Einordnung KI

- Teilgebiete:
 - Maschinelles Beweisen
 - Spielprogramme
 - Expertensysteme
 - Robotik
 - Neuronale Netze
 - Verarbeitung natürlicher Sprache
- Lernen -> Maschinelles Lernen
- Entwicklung von wissensbasierten Systemen
- Wissensverarbeitung -> Wissensrepräsentation



Grundsatzfrage: Lernen?!?

- Wie lernt ein Mensch?
- Wie lernt eine Maschine?
- Welche Aufgaben kann ein Mensch besser?
- Welche Aufgaben sind geeigneter für Maschinen?



Maschinelles Lernen vs. Statistik



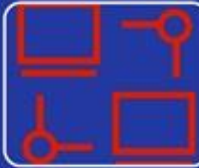
- Kritiker werfen gerne die Frage auf, ob die ML-Prozesse wirklich so revolutionär sind oder nur grundlegende Mathematik und Statistik verwenden. Zweifellos gehen maschinelles Lernen und Datenwissenschaft Hand in Hand, aber was unterscheidet ML von der Statistik, wenn sie beide Datenmuster verwenden, um die Wahrscheinlichkeit oder Unwahrscheinlichkeit verschiedener Situationen aufzuzeigen?
- Es ist leider so, dass der Marketinghype die Realität und den praktischen Nutzen seit Jahrzehnten übertrifft. Die Kombination aus schnellerer Hardware, besseren Modellen und einem besseren Verständnis, wofür maschinelle Lernsysteme geeignet sind, führt oft auch dazu, dass Softwareanwendungen, die letztlich nur Basis-Statistik verwenden, gerne als "Artificial Intelligence-Driven" vermarktet werden. All das macht natürlich eine Beurteilung der Technologie und die Frage nach ihren Auswirkungen sehr schwierig. Prinzipiell muss ja nicht alles mittels maschinellen Lernens gelöst werden, wenn es auch (einfacher) mittels Statistik geht.

Quelle: <https://www.security-insider.de/ki-und-ml-in-der-cybersecurity-a-921089/>

Maschinelles Lernen vs. Statistik

Systeme des maschinellen Lernens und der künstlichen Intelligenz sollten das letzte Mittel sein, das nur dann eingesetzt werden sollte, wenn traditionelle Organisationsmethoden, Musterabgleiche und Statistiken nicht ans Ziel geführt haben.

Machine Learning vs. Statistical Modeling



Machine Learning: Rely on data and algorithms

- Large amount of labeled data
- Feature engineering / log embedding
- Proper learning algorithms

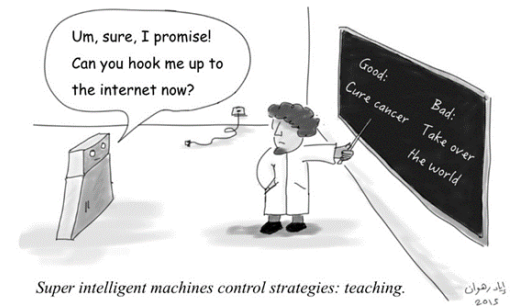


Statistical Modeling: Rely on human's experiences

- Find common trait of attack behavior
- Feature engineering
- Proper statistical algorithms

Quelle: <https://nsfocusglobal.com/practices-of-chinas-internet-giants-in-machine-learning/>

ML & KI in der CyberSecurity



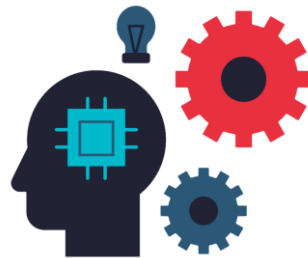
- Künstliche Intelligenz spielt in der Cybersecurity eine immer wichtigere Rolle, indem sie durch **schnelle Analyse** grosser Datenmengen Anomalien und Bedrohungen in Echtzeit erkennt und automatisierte Reaktionen ermöglicht. Gleichzeitig **erhöht sie die Effizienz** von Sicherheitsteams, birgt aber auch Risiken, da auch Angreifer KI-Techniken zur Umgehung von Schutzmassnahmen nutzen können.
- Auch wenn die Marketingbotschaften und teilweise die Berichterstattung etwas anderes suggerieren mag: ***Maschine Learning ist kein Security-Allheilmittel, kann aber den Sicherheits- und IT-Teams viel Unterstützung bieten.***

Aufgaben für ML in CyberSecurity

Es gibt viele Aufgaben für ML die sie bewältigen kann um die Sicherheit zu erhöhen. Dazu gehören:

- **Klassifizierung:** Programme klassifizieren Daten basierend auf vorgegebenen Parametern.
- **Clustering:** Für Daten, die nicht zu den voreingestellten Parametern passen, gruppiert ML-Daten basierend auf ihren Ähnlichkeiten oder Anomalien.
- **Empfehlungen:** Programme lernen aus früheren Entscheidungen, Inputs und Assoziationen, um Ansätze und Entscheidungen zu empfehlen.
- **Generative Frameworks:** Basierend auf früheren Dateneingaben generieren Programme Möglichkeiten, die auf Daten angewendet werden können, die bislang noch nicht in diesem Kontext einbezogen wurden (siehe ChatGPT, Claude, etc.).
- **Prognosen:** Programme erstellen Vorausschauen auf der Grundlage von Daten und vergangenen Ereignissen.

Konkreter:



Cybersicherheits-Aufgaben welche durch den Einsatz von ML vereinfacht und verbessert werden können:

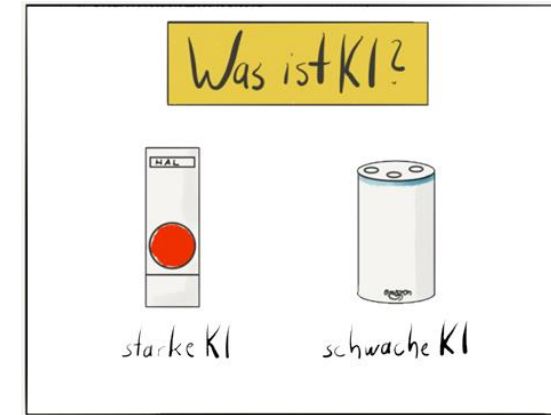
- **Phishing- und Spam-Filterung:** ML kann ihrem Team helfen, interne Daten, Spam und bösartige Aktivitäten mit vorgegebenen Parametern entsprechend zu **klassifizieren**.
- **Forensische Analyse:** ML hilft mittels Clustering, Aufschluss über die Angriffsmethode und Angriffsziele zu erlangen. **Clustering sammelt Daten und gruppiert** diese, wodurch das Team die Elemente des Angriffs einfacher zusammensetzen kann.
- **Incident Response und Risikomanagement:** Durch ML lernt die Software aus den verschiedenen Aktionen, die bei verschiedenen Arten von Vorfällen durchgeführt werden. ML kann **Muster und Assoziationen erkennen** und darauf basierend Empfehlungen zur Risikominderung und zur Reaktion auf Vorfälle abgeben.
- **Penetration Testing:** Ähnlich wie ein Pentester nach Schwachstellen suchen würde, können Programme Eingaben generieren, um Systeme auf Schwachstellen zu testen.
- **Präventions- und Bedrohungsmodellierung:** Durch ML werden Daten gesammelt und analysiert, um betrügerische Aktivitäten vorherzusagen. Dies hilft den Sicherheitsteams, Gefahren zu erkennen, bevor sie zu einem schwerwiegenden und teuren Datenschutzverstoß oder Datendiebstahl werden.

Vorteile und Einsatzgebiete

- Cybersicherheit muss angesichts gesetzlicher Vorgaben wie der DSGVO oder PCIDSS und immer raffinierteren Angriffen zur Top-Priorität von Unternehmen werden.
- Datenschutzverletzungen, Datendiebstähle und andere Angriffe werden immer häufiger und können zu enormen finanziellen Schäden und Geschäftsausfällen führen. Abhängig von der Grösse und den Anforderungen eines Unternehmens kann ML-basierte Sicherheitssoftware eine sinnvolle Nutzung des eines Teils des Cybersicherheitsbudgets sein.
- Zahlreiche sicherheitskritische Prozesse können dank ML wesentlich schneller und präziser durchgeführt werden als es durch eine manuelle Bearbeitung der Fall wäre. So sind fortschrittliche Datensicherheitslösungen in der Lage, einen "Normalzustand" für jeden Benutzer in einem Unternehmen zu erstellen, indem sie die Arten von Daten, auf die er zugreift, abbildet und überwacht.
- Nach einer gewissen Zeit erkennt das System automatisch, wenn jemand keinen Zugang mehr zu sensiblen Informationen benötigt und empfiehlt den Entzug der Zugriffsrechte. Die Software entwickelt diese Empfehlungen nicht nur, weil der spezifische Nutzer diese Daten nicht verwendet hat, sondern bezieht auch Vergleiche mit anderen Nutzern mit ein.

Grundlagen KI

- Was ist KI? Es ist schwer, das eindeutig zu erklären. Grundsätzlich lässt sich sagen, dass es **keine allgemeingültige Definition von Künstlicher Intelligenz** gibt – weil auch der Intelligenz-Begriff nicht eindeutig definiert ist.



Im Deutschen wird gern zwischen starker KI und schwacher KI unterschieden, wenn es um die Definition von KI geht. Einfach erklärt:

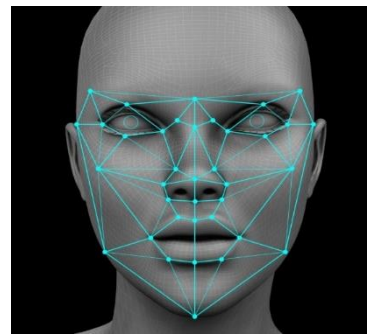
- ✓ **Starke KI** meint das, was wir aus der *Science-Fiction* kennen. Eine Maschine, die Probleme genereller Art lösen kann – also jede Frage, die man ihr stellt. Sie ist bisher noch reine Fantasie und wird es auch über die nächsten Jahrzehnte oder Jahrhunderte bleiben.
- ✓ Mit **schwacher KI** haben wir es hingegen im Alltag zu tun: Das sind Algorithmen – und nichts anderes ist eine KI, ein sehr komplexer Algorithmus – die speziellen Fragen beantworten können, deren Lösungswege sie vorher selbstständig erlernt hat. Sie hat kein eigenes Bewusstsein und zeigt kein Verständnis.

Was macht KI aus?

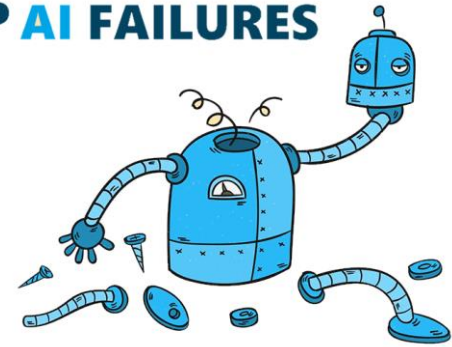


- Was unterscheidet nun eine KI von einem simplen Programm? Üblicherweise schreibt ein Programmierer Code in einer Sprache ihrer Wahl, die aus einem Satz an beliebig komplexen Anweisungen besteht:
 - Wenn dies, dann das
 - Wenn der User auf "Senden", drückt, schicke die E-Mail an den Server XEin solches System nennt man auch **regelbasiert**.
- Bei einer künstlichen Intelligenz gibt der Programmierer nun nicht jeden einzelnen Schritt vor, sondern schreibt einen Algorithmus, der selbstständig in der Lage ist, diese Schritte zu erstellen. Warum ist das wichtig? Weil bestimmte Probleme so kompliziert sind, dass es praktisch unmöglich ist, dafür einen Code zu schreiben.

Beispiel: Gesichtserkennung



- Beispielsweise die Bildererkennung die in sozialen Medien wie Facebook benutzt wird: Kein Programmierer auf der Welt kann einen Satz an Anweisungen schreiben, der immer erkennt wie man aussieht, ganz gleich ob das Foto nachts, am Strand oder im Auto aufgenommen wurde – in einem regelbasierten System wäre das vollkommen unmöglich, denn dazu müsste der Programmierer alle Fälle im Vornherein kennen und mühsam eintippen.
 - Eine KI kennt auch nicht jedes Bild von mir, aber sie kann aus einer Anzahl an vorhandenen Bildern lernen, wie man aussieht und diese Regel dann auf neue Bilder übertragen und einem erkennen. Und das nicht nur mit mir, sondern mit Milliarden Gesichtern in Bruchteilen von Sekunden. Eine KI ist also in der Lage, mit bisher unbekannten Daten umzugehen, Muster zu finden oder Handlungen daraus abzuleiten. Sie lernt eigenständig aus den ihr vorliegenden Daten.
- Was sie lernt, wird dabei aber vom Menschen vorher bestimmt, indem dieser die KI designt. Sie sind damit weitaus mächtiger als regelbasierte Systeme, da sie, mit gewissen Einschränkungen, auf bisher unbekannte Situationen reagieren können und aus Erfahrung lernen.



Was kann KI nicht?

Die KI ist kein genereller Problemlöser – noch nicht. Sie kann zwar Daten ungeheuer gut verarbeiten und Muster erkennen, aber verstehen kann sie sie nicht. Die künstliche Intelligenz besitzt kein "common sense" – keinen Verstand. Wenn sie, aufgrund von **unzureichenden Daten** oder **schlechter Programmierung**, zu falschen Schlüssen kommt, erkennt sie dies nicht.

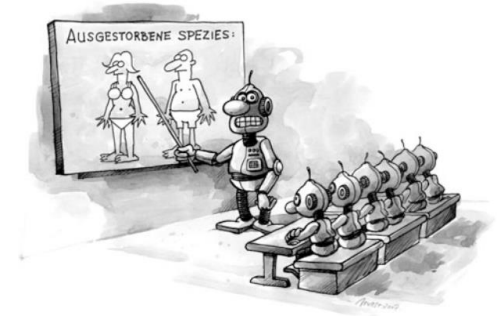
→ Sie kann nur Antworten auf die spezifischen Fragen geben, für welche sie programmiert bzw. trainiert wurde.

Beispiele wo KI versagt hat:

- <https://www.cio.com/article/190888/5-famous-analytics-and-ai-disasters.html>
- <https://www.evidentlyai.com/blog/ai-failures-examples>
- <https://www.immuniweb.com/blog/top-10-failures-of-ai.html>
- <https://syncedreview.com/2021/01/01/2020-in-review-10-ai-failures/>

Grundlagen ML

Machine Learning ist ein Teilbereich der KI



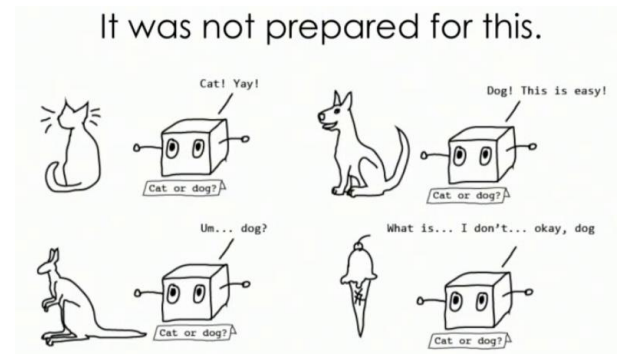
Mithilfe des maschinellen Lernens werden IT-Systeme in die Lage versetzt, auf Basis vorhandener Datenbestände und Algorithmen Muster und Gesetzmässigkeiten zu erkennen und Lösungen zu entwickeln/formulieren.

Es wird quasi künstliches Wissen aus Erfahrungen generiert. Die aus den Daten gewonnenen Erkenntnisse lassen sich verallgemeinern und für neue Problemlösungen oder für die Analyse von bisher unbekannten Daten verwenden.

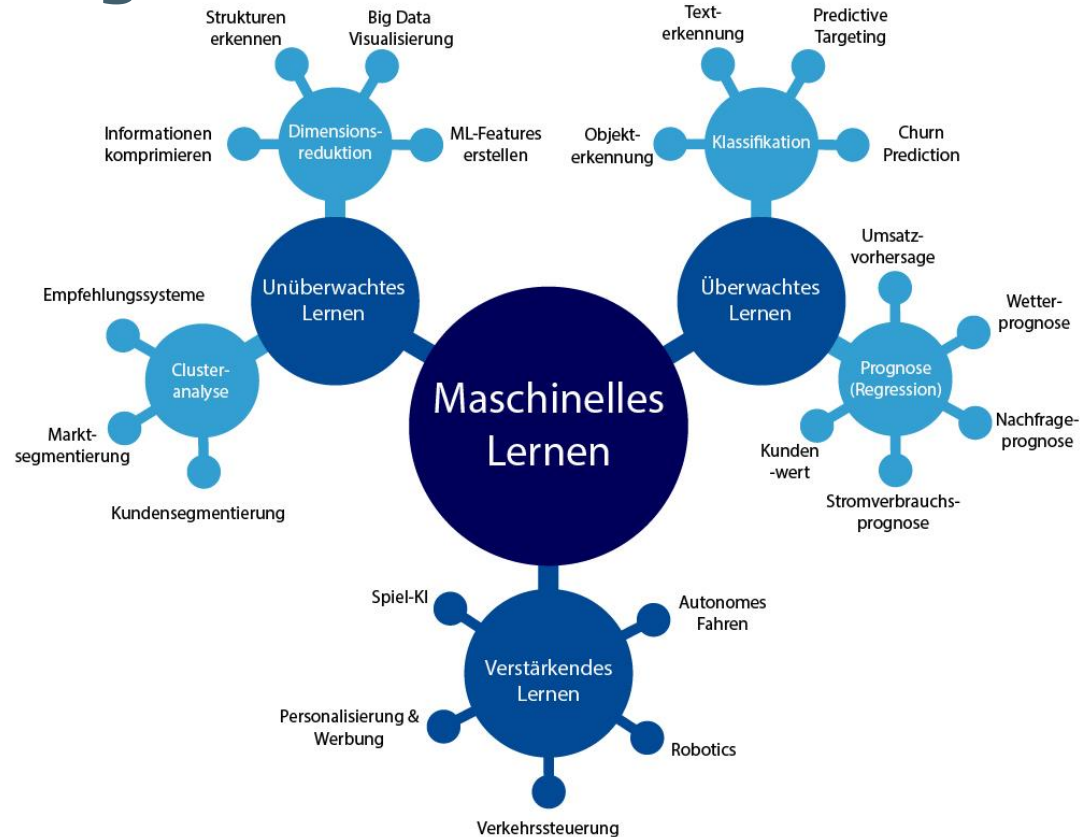
Quelle: <https://www.bigdata-insider.de/was-ist-machine-learning-a-592092/>

Grundlagen ML (2)

- Damit die Software eigenständig lernen und Lösungen finden kann, ist ein vorheriges Handeln von Menschen notwendig.
- Beispielsweise müssen die Systeme zunächst mit den für das Lernen relevanten Daten und Algorithmen versorgt werden. Zudem sind Regeln für die Analyse des Datenbestands und das Erkennen der Muster aufzustellen. Sind passende Daten vorhanden und Regeln definiert, können Systeme mit maschinellem Lernen unter anderem folgendes:
 - Relevante Daten finden, extrahieren und zusammenfassen,
 - Vorhersagen auf Basis der analysierten Daten treffen,
 - Wahrscheinlichkeiten für bestimmte Ereignisse berechnen,
 - sich an Entwicklungen eigenständig anpassen und
 - Prozesse auf Basis erkannter Muster optimieren.



Machine Learning Übersicht



Arten von ML

- Algorithmen nehmen beim maschinellen Lernen eine zentrale Rolle ein. Sie sind für das Erkennen von Mustern und das Generieren von Lösungen verantwortlich und lassen sich in verschiedene Lernkategorien einteilen:
 - überwachtes Lernen (Supervised Machine Learning)
 - unüberwachtes Lernen (Unsupervised Machine Learning)
 - teilüberwachtes Lernen
 - bestärkendes Lernen
 - aktives Lernen
- Während beim **überwachten Lernen** im Vorfeld Beispielmuster definiert und spezifiziert werden müssen, um die Informationen passend den Modellgruppen der Algorithmen zuzuordnen, werden die Modellgruppen beim **unüberwachten Lernen** automatisiert aufgrund eigenständig erkannter Muster gebildet.

Quelle: <https://www.bigdata-insider.de/was-ist-machine-learning-a-592092/>

Arten von ML (2)

- **Teilüberwachtes Lernen** stellt eine Mischung aus beiden Methoden dar.
- Das **bestärkende Lernen** basiert auf Belohnungen und Bestrafungen. Dem Algorithmus wird durch diese Interaktion mitgeteilt, wie er auf verschiedene Situationen zu reagieren hat. Diese Lernweise ist dem menschlichen Lernen sehr ähnlich.
- **Aktives Lernen** schliesslich bietet dem Algorithmus die Möglichkeit, für bestimmte Eingangsdaten die gewünschten Ergebnisse zu erfragen. Um die Anzahl von Fragen zu minimieren, erfolgt zuvor eine Auswahl relevanter Fragen mit hoher Ergebnisrelevanz durch den Algorithmus selbst.

Abhängig vom jeweiligen System kann die Datenbasis offline oder online vorliegen und wiederholbar oder nur einmalig für das maschinelle Lernen zur Verfügung stehen. Ein weiteres Unterscheidungsmerkmal des Machine Learnings ist das gleichzeitige Vorhandensein der Ein- und Ausgabe-Paare oder deren zeitlich versetzte Entwicklung. Je nach Art spricht man vom Batch-Lernen oder vom sequenziellen Lernen.

Quelle: <https://www.bigdata-insider.de/was-ist-machine-learning-a-592092/>

ML-Methoden

- Obwohl es eine grosse Anzahl an Machine Learning Methoden und Aufgaben gibt die in eine dieser beiden Kategorien (supervised bzw. unsupervised) von Algorithmen fallen, lässt sich doch ein kleines Set von Standards herausstellen.
- Im Fall von *Supervised Machine Learning* gehören **Classification** und **Regression** zu den Standard-Methoden
- Im Fall von *Unsupervised Machine Learning* sind **Clustering** und **Frequent Patterns** häufig eingesetzte Methoden.

Classification und Regression

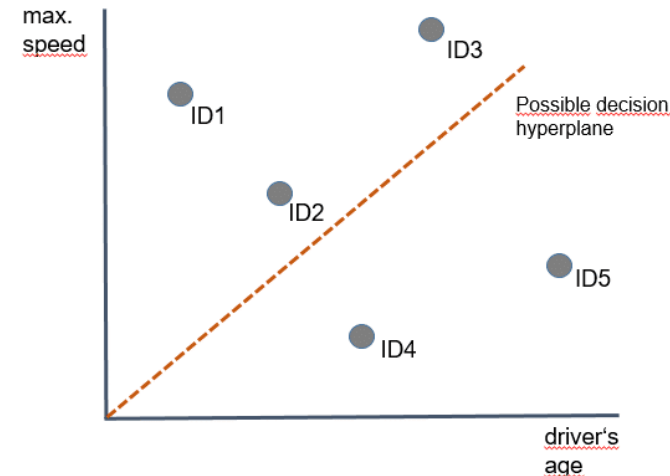
- Um die Bedeutung der Machine Learning Methoden wie *Classification* und *Regression* richtig zu erfassen, ist es wichtig sich vor Augen zu halten, dass diese die Grundelemente dessen darstellen, was unter den Begriff der Künstlichen Intelligenz gefasst wird.
- Die mathematischen bzw. statistischen Verfahren helfen intelligenten Systemen dabei, das **Ordnen von Dingen und Ereignissen** zu lernen. Sie machen dies selbstverständlich nicht bewusst, sondern wie in diesem Fall mit der Hilfe von *Data Scientists*, die die Lernvorgänge überwachen.
- Es gibt nicht den einen Algorithmus beziehungsweise Verfahren, um alle Classification-Aufgaben zu lösen. Es gibt vielmehr eine grosse Vielfalt an Unter- und Sonderformen. Viele von ihnen haben einen Ursprung in der Mathematik bzw. Statistik.

Classification und Regression (1)

z.B. Lineare Classification (Beispiel dazu mit Logistic Regression kommt noch)

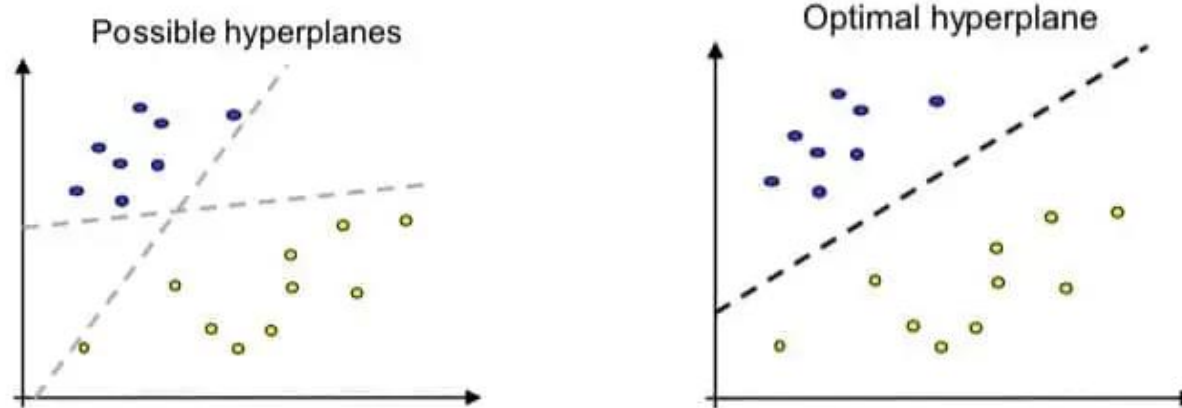
- Die Lineare Classification – manchmal auch als "Linear Classifier" bezeichnet – zielt darauf ab, eine ganz bestimmte, lineare Funktion zu bestimmen. Diese Funktion **beschreibt eine Grenze**, die Daten in zwei Klassen unterteilt:

- Beschreibung: Die Daten aus der Tabelle werden in ein Koordinatensystem übertragen, wo eine lineare Funktion sie in zwei Klassen aufteilt.



Classification und Regression (2)

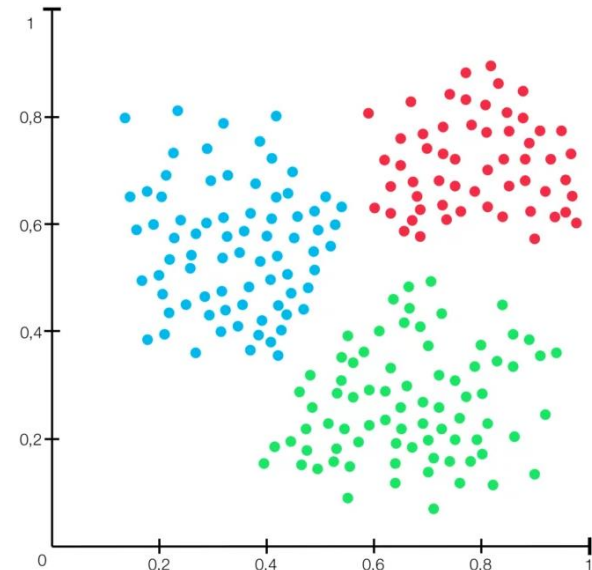
Was sich zunächst nach einer sehr einfachen Methode anhört, findet in zahlreichen und teilweise komplexen Variationen Anwendung. Zu den beliebtesten Linear Classifiers zählen Support Vector Machines. Diese werden angewandt, um Daten in einem Vektorraum in zwei unterschiedliche Klassen zu unterteilen. Die "Kunst" bei der linearen Klassifikation besteht darin, den *Linear Classifier* so optimal wie möglich zu definieren.



Quelle: <https://www.alexanderthamm.com/de/blog/machine-learning-ultimate-ratgeber/>

Clustering

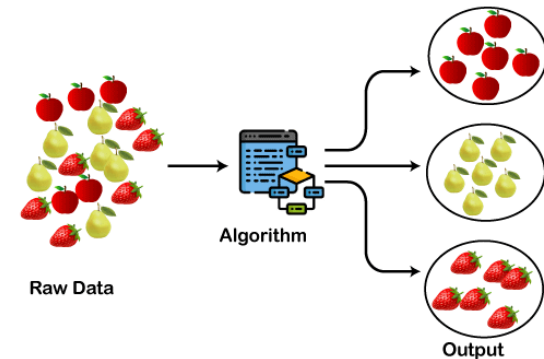
- Unter dem Clusteringproblem versteht man das Zusammenfassen von Objekten oder Daten zu Gruppen, den Clustern. Sie werden im Gegensatz zur Classification nicht vorgegeben, sondern ergeben sich auf Grund der Ähnlichkeit der verschiedenen, betrachteten Objekte. Mit Ähnlichkeit ist dabei der räumliche Abstand der Objekte, als Vektoren dargestellt, gemeint.
- Es gibt verschiedene Verfahren diesen Abstand zu bestimmen wie z.B. die Euklidische- oder Minkowski-Metrik.



Quelle: <https://rocketloop.de/de/blog/clustering-mit-machine-learning/>

Clusteringverfahren

- Es gibt verschiedene Verfahren die einzelnen Objekte in Cluster einzuordnen, die jeweils ihre eigenen Vor- und Nachteile haben und andere Ergebnisse erzielen.
- Meistens muss man, um die besten Ergebnisse für einen Datensatz zu erzielen, verschiedene dieser Verfahren mit unterschiedlichen Parametern ausprobieren und sich danach für eines entscheiden. Es existiert nämlich kein Verfahren, dass allgemein am besten funktioniert.
- Zu den häufig vorkommenden *Unsupervised Learning* Verfahren zählen:
 - Hierarchisches Clustering
 - Partitionierendes Clustering
 - Dichtenbasiertes Clustering



Quelle: <https://rocketloop.de/de/blog/clustering-mit-machine-learning/>

Clusteringverfahren (2)

Hierarchisches Clustering

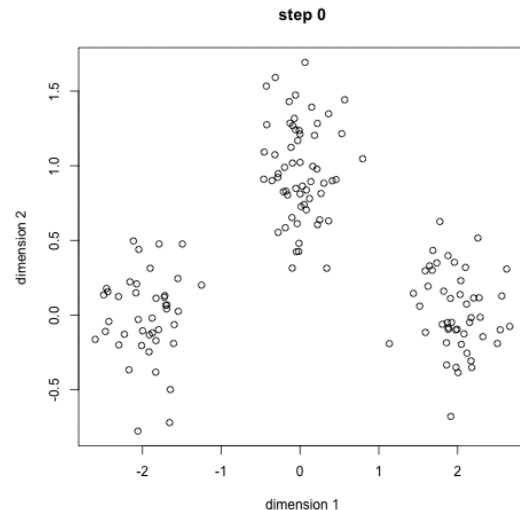
- Beim hierarchischen Clustering wird zu Beginn entweder die Menge aller Objekte gesamt (**top-down**) oder die Menge aller einzelnen Objekte (**bottom-up**) als Cluster angenommen.
- Beim **top-down** Verfahren, wird der jeweils grösste Cluster aufgeteilt, in dem das am weitesten Entfernte Objekt zum Zentrum des Clusters als ein neues Clusterzentrum ausgewählt wird. Danach ordnet der Algorithmus alle Objekte dem jeweils näheren Clusterzentrum zu.
- Beim **bottom-up** Verfahren werden die jeweils kleinsten und nahesten Cluster zu einem neuen Cluster zusammengefasst.
- Schwierig bei beiden Verfahren ist, einerseits festzulegen auf welche Art man den Abstand der einzelnen Objekte bestimmt, andererseits, wie weit man die Cluster aufteilen, bzw. zusammenfassen will. Schliesslich sollte man den Vorgang nicht so lange wiederholen, bis so viele Cluster wie Objekte oder nur noch ein Cluster übrig bleiben.

Quelle: <https://rocketloop.de/de/blog/clustering-mit-machine-learning/>

Clusteringverfahren (3)

Partitionierendes Clustering

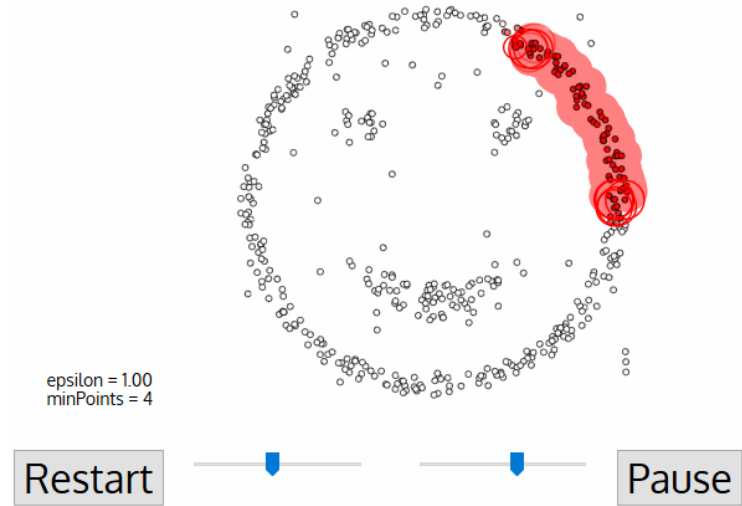
- Beim partitionierenden Clustering wird zunächst eine Zahl k festgelegt, die die Anzahl der Cluster vorgibt. Darauf bestimmt der jeweilige Algorithmus k Clusterzentren.
- Beispielsweise legt der **K-Means-Algorithmus**, einer der verbreitetsten partitionierenden Clusteringverfahren, die Clusterzentren zufällig fest.
- Diese Zentren werden anschliessend so verschoben, dass die *Summe der quadrierten Abstände der Objekte zu ihrem nächsten Clusterzentrum minimiert wird*.
- Vorteil des partitionierenden Verfahrens ist, dass die Zugehörigkeit der einzelnen Objekte zu einem Cluster während der Clusterfindung veränderbar ist. Andererseits ist die Eigenschaft, dass bereits zu Beginn die Anzahl der Clusterzentren festgelegt wird ein grosser Nachteil, da man um optimale Ergebnisse zu erzielen, den Algorithmus mit verschiedenen Werten ausführen und anschliessend eines auswählen muss.



Clusteringverfahren (4)

Dichtenbasiertes Clustering

- Beim dichte-basierten Clustering werden die verschiedenen Cluster durch die Dichte der einzelnen Objekte im Raum bestimmt.
- Das populärste dieser Verfahren ist der DBSCAN Algorithmus. Dabei wird ein Abstand ϵ zusammen mit einer Zahl k bestimmt. Objekte, die k Nachbarobjekte mit dem Abstand kleiner ϵ besitzen, stellen zusammen Kernobjekte dar.
- Wenn zwei Kernobjekte mit Abstand ϵ zueinander vorliegen, werden sie zu einem Cluster zusammengefasst.
- Objekte die keine Kernobjekte sind, sich aber in der Nähe eines Clusters befinden, ordnet man dem Cluster hinzu. Die Objekte die keinem Cluster zugehörig sind, gelten als Rauschobjekte.



Quelle: <https://rocketloop.de/de/blog/clustering-mit-machine-learning/>

Bildquelle: <https://towardsdatascience.com/the-5-clustering-algorithms-data-scientists-need-to-know-a36d136ef68>

Schwierigkeiten beim Clustering

- Das Clustering ist bei grossen Datenmengen sehr zeitaufwendig, besonders wenn der Raum mehrere Dimensionen besitzt.
- Weiter ist die Effektivität des Clusterings durch das gewählte Verfahren der Abstandmessung bestimmt. In der Regel gibt es kein allgemein richtiges Ergebnis. Das liegt daran, dass dieses in Abhängigkeit der verschiedenen Verfahren mit jeweils verschiedenen Parametern zum Teil drastisch unterschiedlich ausfällt. Dadurch ist es schwer, das Ergebnis zu interpretieren und die Richtigkeit zu überprüfen.
- Möglichkeiten diese Überprüfung durchzuführen wären, das Resultat des Clusterings mit bekannten Klassifizierungen zu vergleichen oder die Auswertung durch einen menschlichen Experten.
- Beide Varianten sind aber nur bedingt geeignet, da einerseits das Clustering nicht nötig wäre, wenn es bereits bekannte Klassifizierungen gäbe und andererseits menschliche Beurteilungen stets sehr subjektiv ist.

Beispiele

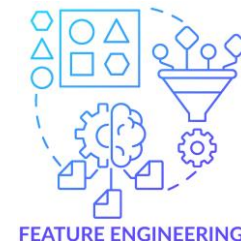
- ML hat ein sehr grosses Spektrum an Anwendungsmöglichkeiten. Im Internet- bzw. im CyberSecurity-Umfeld kommt maschinelles Lernen beispielsweise für folgende Funktionen zum Einsatz:
 - selbstständiges Erkennen von Spam-Mails und Entwicklung geeigneter Spam-Filter
 - Erkennung von potentiell schädlichen URLs
 - Erkennung und Unterscheidung der Internetaktivität von natürlichen Personen und Bots (Malware?)
 - Weitere Anwendungsbereiche des maschinellen Lernens sind z.B. die automatische Erkennung von Kreditkartenbetrug
 - ...

Auswahl der Methoden

- Etwas vom Schwierigsten beim Einsatz von ML-Technologien ist die Wahl der Methode bzw. die Auswahl des einzusetzenden Algorithmus.
- Dazu braucht es viel Erfahrung und auch ein wenig Fingerspitzengefühl und ggf. die Unterstützung eines trainierten Data-Analytikers
- Helfen können auch Zusammenstellungen der Methoden und Algorithmen mit ihren Charakteristiken wie z.B. :
<https://docs.microsoft.com/en-us/azure/machine-learning/algorithm-cheat-sheet>
oder: <https://stanford.edu/~shervine/teaching/cs-229/cheatsheet-machine-learning-tips-and-tricks#>
oder: <https://ape-ai.com/books/ML/Machine%20Learning%20Interview%20Cheat%20sheets.pdf>

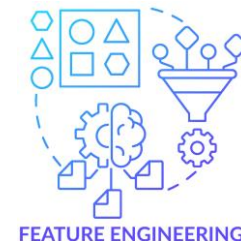
Die Datenbasis

- Alle auf KI und ML basierenden Verfahren sind abhängig von ihrer Datenbasis. Nur wenn diese gross genug ist lassen sie sich effektiv einsetzen.
- Ausserdem ist auch die Qualität der Daten von grosser Wichtigkeit. Fehlende bzw. unvollständige Daten könnten einen merkbaren Einfluss auf die Ergebnisse haben.
- Ggf. müssen die Daten aufbereitet werden bevor sie an ML-Systeme übergeben werden können.
- Schlussendlich ist das **Feature Engineering** das A und O für den guten Einsatz von KI-Systemen



Feature Engineering (1)

- Feature Engineering ist ein entscheidender Schritt im Bereich des maschinellen Lernens und der künstlichen Intelligenz (KI). Dabei geht es darum, **Rohdaten in Merkmale (Features) umzuwandeln**, die für das Training von Machine-Learning-Modellen verwendet werden können.
- Hier sind einige wichtige Aspekte des Feature Engineering:
 1. Datenaufbereitung: Rohdaten werden bereinigt und in ein Format gebracht, das für Algorithmen verständlich ist.
 2. Merkmalsauswahl: Relevante Merkmale werden identifiziert und ausgewählt, um die Leistung des Modells zu verbessern.
 3. Merkmalstransformation: Bestehende Merkmale werden transformiert oder neue Merkmale werden erstellt, um die Informationsdichte zu erhöhen.
 4. Domänenwissen: Fachwissen wird genutzt, um Merkmale zu generieren, die spezifisch für das Problem sind.

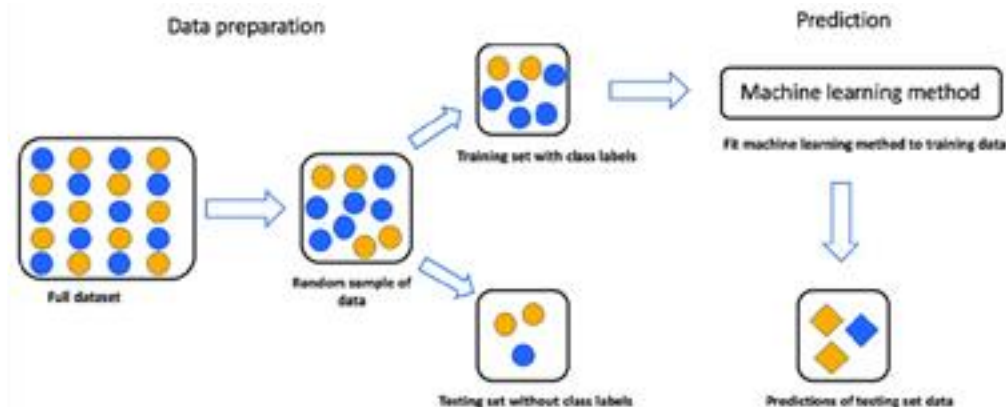


Feature Engineering (2)

- Ein gutes Feature Engineering kann die Genauigkeit und Effizienz von Machine-Learning-Modellen erheblich verbessern, indem es die relevanten Informationen aus den Daten extrahiert und unnötige oder redundante Daten entfernt.
- Ein gutes Beispiel ist die Erkennung von Kreditkartenbetrug.
Hier könnten Merkmale wie Transaktionsbetrag, Ort der Transaktion, Zeitstempel und Häufigkeit der Transaktionen innerhalb eines bestimmten Zeitraums verwendet werden, um verdächtige Aktivitäten zu identifizieren.
- Weitere Beispiele, hier aus dem Bereich E-Commerce:
 - **Kundenverhalten:** Analyse des Kaufverhaltens, z.B. Häufigkeit der Käufe, durchschnittlicher Bestellwert, bevorzugte Kategorien.
 - **Produktmerkmale:** Erstellung von Merkmalen wie Beliebtheit eines Produkts, Bewertungen und Rezensionen, Lagerbestand.

Datenaufbereitung (1)

Die Datenaufbereitung (auch als Data Preparation, Data Pre-Processing, Datenvorverarbeitung, etc. bezeichnet) ist der Prozess der Umwandlung von Rohdaten, damit Datenwissenschaftler und Analysten sie durch Algorithmen des ML laufen lassen können, um Erkenntnisse aufzudecken oder Vorhersagen zu treffen.



C	D
age	weight
[50-60)	?
[20-30)	[50-75)
[80-90)	?
[50-60)	?
[50-60)	?
[70-80)	?

Datenaufbereitung – Probleme (1)

Der Datenaufbereitungsprozess kann durch folgende Probleme erschwert werden:

- **Fehlende oder unvollständige Datensätze.** Es ist schwierig, jeden Datenpunkt für jeden Datensatz in einem Dataset zu erhalten. Fehlende Daten erscheinen manchmal als leere Zellen, Werte (z. B. NULL oder N/A) oder ein bestimmtes Zeichen, wie z. B. ein Fragezeichen.
- **Ausreisser oder Anomalien.** Unerwartete Werte tauchen oft in einer Werteverteilung auf, vor allem, wenn Sie mit Daten aus unbekannten Quellen arbeiten, die nur unzureichende Datenvalidierungskontrollen aufweisen.
- **Unsachgemäss formatierte / strukturierte Daten.** Daten müssen manchmal in ein anderes Format oder an einen anderen Ort extrahiert werden. Eine gute Möglichkeit, dies zu beheben, besteht darin, Fachexperten zu konsultieren oder Daten aus anderen Quellen zusammenzuführen.

Datenaufbereitung – Probleme (2)

- **Inkonsistente Werte** und nicht standardisierte kategoriale Variablen.
Beim Kombinieren von Daten aus mehreren Quellen kann es zu Abweichungen bei Variablen wie Firmennamen oder Staaten kommen. Zum Beispiel könnte ein Bundesland in einem System "Texas" sein, während es in einem anderen "TX" sein könnte. Das Auffinden aller Variationen und die korrekte Standardisierung verbessern die Modellgenauigkeit erheblich.
- **Die Notwendigkeit von Techniken wie dem Feature Engineering.**
Selbst wenn alle relevanten Daten verfügbar sind, kann der Datenaufbereitungsprozess Techniken wie das Feature Engineering erfordern, um zusätzliche Inhalte zu generieren, die zu genaueren, relevanten Modellen führen.

Datenaufbereitung - Probleme (3)

- **Begrenzte oder spärliche Merkmale/Attribute.**

Die Anreicherung von Merkmalen oder der Aufbau von Merkmalen in den Daten erfordert oft, dass man Datensätze aus verschiedenen Quellen kombinieren. Das Zusammenführen von Dateien aus verschiedenen Systemen wird oft erschwert, wenn es **keine einfachen** oder **exakten Spalten** gibt, um die Datensätze abzugleichen.

- Dies erfordert dann die Fähigkeit, ein Fuzzy-Matching durchzuführen welches auch auf der Kombination mehrerer Spalten basieren kann um die Übereinstimmung zu erreichen. Zum Beispiel könnte es einfach sein, zwei Datensätze über die KUNDEN-ID zu kombinieren (die in beiden Datensätzen vorhanden ist). Das Kombinieren eines Datensatzes welcher separate Spalten für KUNDENVORNAME und KUNDENNACHNAME hat, mit einem anderen Datensatz mit einer Spalte KUNDENVOLLNAME, die "Nachname, Vorname" enthält, wird schwieriger.

Wichtigkeit der Datenaufbereitung (1)

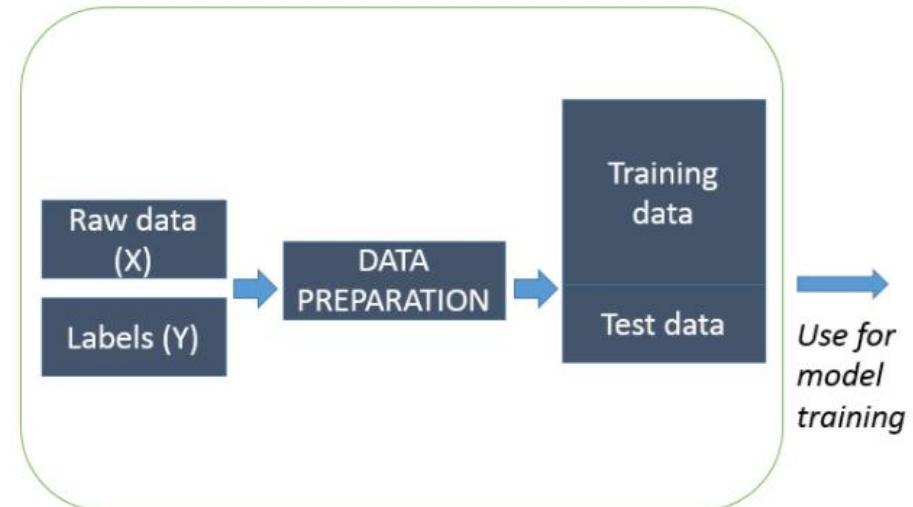
Die meisten Algorithmen für maschinelles Lernen erfordern, dass die Daten auf eine ganz bestimmte Art und Weise formatiert werden, sodass die Datensätze im Allgemeinen ein gewisses Mass an Vorbereitung erfordern, bevor sie nützliche Erkenntnisse liefern können.

- Einige Datensätze enthalten Werte, die fehlen, ungültig sind oder anderweitig für einen Algorithmus schwierig zu verarbeiten sind.
- Wenn Daten fehlen, kann der Algorithmus sie nicht verwenden.
- Wenn Daten ungültig sind, liefert der Algorithmus weniger genaue oder sogar irreführende Ergebnisse.
- Einige Datensätze sind relativ sauber, müssen aber geformt werden (z. B. aggregiert oder ge-pivotet),
- und vielen Datensätzen fehlt es einfach an nützlichem Geschäftskontext (z. B. schlecht definierte ID-Werte)

daher ist eine Anreicherung mit Funktionen erforderlich. Eine gute Datenvorbereitung erzeugt saubere und gut kuratierte Daten, die zu praktischeren, genaueren Modellergebnissen führen.

Wichtigkeit der Datenaufbereitung (2)

- Die Datenaufbereitung wird von einigen Fachleuten als wichtigster Schritt beim Trainieren von ML- Systemen betrachtet
- Die Datenvorbereitung umfasst mehrere Ideen:
 - Datenbereinigung
 - Daten-Suffizienz
 - Merkmalsauswahl
 - Merkmalsextraktion
 - Merkmalsskalierung
 - Stichprobenziehung
 - Datensätze
 - Dimensionalitätsreduktion



Beispiel: Network Anomaly Detection

- Das folgende Beispiel zeigt exemplarisch, wie man mittels Python und der *Logistic Regression** Methode ein Modell trainiert, um zu erkennen, ob der gemessene Traffic einen Angriff darstellt oder nicht.
- Das Datenset kommt von:
<https://www.kaggle.com/datasets/anushonkar/network-anamoly-detection>
- Bearbeitet wurde es unter Google Colab:
https://colab.research.google.com/drive/1snULVYFH9xzpOoJ3e_pvsGmbIBQlcwAV?usp=sharing

*

Die logistische Regression ist ein statistisches Verfahren, das die Wahrscheinlichkeit vorhersagt, dass ein bestimmtes Ereignis eintritt, z. B. **ja/nein**, auf der Grundlage unabhängiger Variablen, und wird hauptsächlich für **binäre** Klassifikationen eingesetzt.

▼ Schritt 1: Daten Laden

```
[105] import numpy as np  
import pandas as pd  
  
import seaborn as sns  
import matplotlib as plt
```

Benötigte Informationen
über die Datenmenge

```
columns = ["duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes", "land", "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",  
"num_compromised", "root_shell", "su_attempted", "num_root", "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds", "is_host_login",  
"is_guest_login", "count", "srv_count", "error_rate", "srv_error_rate", "error_rate", "srv_error_rate", "same_srv_rate", "diff_srv_rate",  
"srv_diff_host_rate", "dst_host_count", "dst_host_srv_count", "dst_host_same_srv_rate", "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",  
"dst_host_srv_diff_host_rate", "dst_host_error_rate", "dst_host_srv_error_rate", "dst_host_error_rate", "dst_host_srv_error_rate", "attack", "last_flag"]
```

▼ Schritt 2: Daten Analyse und Exploration

<https://www.kaggle.com/datasets/anushonkar/network-anomaly-detection>

```
✓ [106] data = pd.read_csv("Train.txt", sep=",", names=columns)
```

Daten für das Trainieren des Modells

```
✓ [107] data_test = pd.read_csv("Test.txt", sep=",", names=columns)
```

Daten für das Überprüfen der Berechnung

▼ Schritt 3: Data Pre-processing

✓ [108] data.shape

↗ (125973, 43)

shape: stellt die Dimensionen einer Datenmenge fest, hier: 125973 Zeilen mit 43 Spalten

✓ [109] data_test.shape

↗ (22544, 43)

✓ [110] data.head()

↗

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_port_rate	dst_host_srv
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	0.17	0.03	0.17	
1	0	udp	other	SF	146	0	0	0	0	0	...	0.00	0.60	0.88	
2	0	tcp	private	S0	0	0	0	0	0	0	...	0.10	0.05	0.00	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	1.00	0.00	0.03	
4	0	tcp	http	SF	199	420	0	0	0	0	...	1.00	0.00	0.00	

5 rows × 43 columns

<

>

✓ [] data.describe()

↗

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	dst_host_srv_count	dst_host_s
count	125973.00000	1.259730e+05	1.259730e+05	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	...	125973.000000	1
mean	287.14465	4.556674e+04	1.977911e+04	0.000198	0.022687	0.000111	0.204409	0.001222	0.395736	0.279250	...	115.653005	
std	2604.51531	5.870331e+06	4.021269e+06	0.014086	0.253530	0.014366	2.149968	0.045239	0.489010	23.942042	...	110.702741	
min	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	
25%	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	10.000000	
50%	0.00000	4.400000e+01	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	63.000000	
75%	0.00000	2.760000e+02	5.160000e+02	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	...	255.000000	
max	42908.00000	1.379964e+09	1.309937e+09	1.000000	3.000000	3.000000	77.000000	5.000000	1.000000	7479.000000	...	255.000000	

8 rows × 39 columns

<

>

✓ [165] data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 125973 entries, 0 to 125972
Data columns (total 43 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   duration                                  125973 non-null  int64
1   protocol_type                             125973 non-null  object
2   service                                   125973 non-null  object
3   flag                                       125973 non-null  object
4   src_bytes                                 125973 non-null  int64
5   dst_bytes                                 125973 non-null  int64
6   land                                      125973 non-null  int64
7   wrong_fragment                           125973 non-null  int64
8   urgent                                   125973 non-null  int64
9   hot                                       125973 non-null  int64
10  num_failed_logins                         125973 non-null  int64
11  logged_in                                125973 non-null  int64
12  num_compromised                           125973 non-null  int64
13  root_shell                                125973 non-null  int64
14  su_attempted                              125973 non-null  int64
15  num_root                                  125973 non-null  int64
16  num_file_creations                        125973 non-null  int64
17  num_shells                                125973 non-null  int64
18  num_access_files                          125973 non-null  int64
19  num_outbound_cmds                         125973 non-null  int64
20  is_host_login                             125973 non-null  int64
21  is_guest_login                             125973 non-null  int64
22  count                                      125973 non-null  int64
23  srv_count                                 125973 non-null  int64
24  serror_rate                               125973 non-null  float64
25  srv_serror_rate                           125973 non-null  float64
26  rerror_rate                               125973 non-null  float64
27  srv_rerror_rate                           125973 non-null  float64
28  same_srv_rate                             125973 non-null  float64
29  diff_srv_rate                             125973 non-null  float64
30  srv_diff_host_rate                        125973 non-null  float64
31  dst_host_count                             125973 non-null  int64
32  dst_host_srv_count                         125973 non-null  int64
33  dst_host_same_srv_rate                    125973 non-null  float64
34  dst_host_diff_srv_rate                    125973 non-null  float64
35  dst_host_same_src_port_rate               125973 non-null  float64
36  dst_host_srv_diff_host_rate               125973 non-null  float64
37  dst_host_serror_rate                      125973 non-null  float64
38  dst_host_srv_serror_rate                  125973 non-null  float64
39  dst_host_rerror_rate                      125973 non-null  float64
40  dst_host_srv_rerror_rate                  125973 non-null  float64
41  attack                                    125973 non-null  object
42  last_flag                                 125973 non-null  int64
dtypes: float64(15), int64(24), object(4)
memory usage: 41.3+ MB
```

info:
kompakte
Übersicht
über
Struktur der
Datenmenge

✓ [166] data[['land', 'urgent', 'num_failed_logins', 'num_outbound_cmds']]

	land	urgent	num_failed_logins	num_outbound_cmds
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0
...	...	...	...	...
125968	0	0	0	0
125969	0	0	0	0
125970	0	0	0	0
125971	0	0	0	0
125972	0	0	0	0

125973 rows x 4 columns

✓ [167] data[['land', 'urgent', 'num_failed_logins', 'num_outbound_cmds']].describe()

	land	urgent	num_failed_logins	num_outbound_cmds
count	125973.000000	125973.000000	125973.000000	125973.0
mean	0.000198	0.000111	0.001222	0.0
std	0.014086	0.014366	0.045239	0.0
min	0.000000	0.000000	0.000000	0.0
25%	0.000000	0.000000	0.000000	0.0
50%	0.000000	0.000000	0.000000	0.0
75%	0.000000	0.000000	0.000000	0.0
max	1.000000	3.000000	5.000000	0.0

Werte von diesen 4 Spalten sind praktisch immer 0 -> das bringt keinen Mehrwert und wir können sie für die weitere Analyse ausschliessen

```
[168] data.drop(['land', 'urgent', 'num_failed_logins', 'num_outbound_cmds'], axis=1, inplace=True)
```

```
[169] data.shape
```

```
(125973, 39)
```

Nun kontrolliert man, ob es fehlende Elemente (Werte gibt)

```
[170] data.isna().sum(axis=0)
```

num_compromised	0
root_shell	0
su_attempted	0
num_root	0
num_file_creations	0
num_shells	0
num_access_files	0
is_host_login	0
is_guest_login	0
count	0
srv_count	0
error_rate	0
srv_error_rate	0
error_rate	0
srv_error_rate	0
same_srv_rate	0
diff_srv_rate	0
srv_diff_host_rate	0
dst_host_count	0
dst_host_srv_count	0
dst_host_same_srv_rate	0
dst_host_diff_srv_rate	0
dst_host_same_src_port_rate	0
dst_host_srv_diff_host_rate	0
dst_host_error_rate	0
dst_host_srv_error_rate	0
dst_host_error_rate	0
dst_host_srv_error_rate	0
attack	0
last_flag	0

dtype: int64

sieht gut aus, keine der Felder hat ein fehlendes Element sonst hätte man mit: `clean_data=data.dropna(axis=1)` diese Zeilen entfernen können

drop: löschen von Spalten in der Datenmenge

Als nächstes sind die Felder dran, welche nicht aus Zahlen bestehen

```
[171] data.select_dtypes(exclude=[np.number])
```

	protocol_type	service	flag	attack
0	tcp	ftp_data	SF	normal
1	udp	other	SF	normal
2	tcp	private	S0	neptune
3	tcp	http	SF	normal
4	tcp	http	SF	normal
...	...	...	...	...
125968	tcp	private	S0	neptune
125969	udp	private	SF	normal
125970	tcp	smtp	SF	normal
125971	tcp	klogin	S0	neptune
125972	tcp	ftp_data	SF	normal

125973 rows × 4 columns

```
[172] data.shape
```

```
(125973, 39)
```

unique: mal schauen, welche "nicht-zahlenwerte" die verschiedenen Spalten haben

```
[173] data['attack'].unique()
```

```
array(['normal', 'neptune', 'warezclient', 'ipsweep', 'portsweep',  
       'teardrop', 'nmap', 'satan', 'smurf', 'pod', 'back',  
       'guess_passwd', 'ftp_write', 'multihop', 'rootkit',  
       'buffer_overflow', 'imap', 'warezmaster', 'phf', 'land',  
       'loadmodule', 'spy', 'perl'], dtype=object)
```

```
[174] data['flag'].unique()
```

```
array(['SF', 'S0', 'REJ', 'RSTR', 'SH', 'RSTO', 'S1', 'RSTOS0', 'S3',  
       'S2', 'OTH'], dtype=object)
```

```
[122] data['service'].unique()

0s array(['ftp_data', 'other', 'private', 'http', 'remote_job', 'name',
        'netbios_ns', 'eco_i', 'mtp', 'telnet', 'finger', 'domain_u',
        'supdup', 'uucp_path', 'Z39_50', 'smtp', 'csnet_ns', 'uucp',
        'netbios_dgm', 'urp_i', 'auth', 'domain', 'ftp', 'bgp', 'ldap',
        'ecr_i', 'gopher', 'vmnet', 'systat', 'http_443', 'efs', 'whois',
        'imap4', 'iso_tsap', 'echo', 'klogin', 'link', 'sunrpc', 'login',
        'kshell', 'sql_net', 'time', 'hostnames', 'exec', 'ntp_u',
        'discard', 'nntp', 'courier', 'ctf', 'ssh', 'daytime', 'shell',
        'netstat', 'pop_3', 'nntp', 'IRC', 'pop_2', 'printer', 'tim_i',
        'pm_dump', 'red_i', 'netbios_ssn', 'rje', 'X11', 'urh_i',
        'http_8001', 'aol', 'http_2784', 'tftp_u', 'harvest'], dtype=object)
```

```
[123] data['protocol_type'].unique()
```

```
array(['tcp', 'udp', 'icmp'], dtype=object)
```

An dieser Stelle muss man entscheiden, ob man nur einen einfachen Klassifizierer will -> als Attacke oder keine Attacke. In diesem Beispiel möchten wir nur einen 'binären' Klassifizierer. Deshalb wird alles, was unter 'attack' steht und nicht den Wert 'normal' hat als Attacke eingeordnet

```
[124] np.where(data['attack'] != 'normal')
```

```
array([ 2, 5, 6, ..., 125966, 125968, 125971]),)
```

```
[125] np.where(data['attack'] != "normal", "attack", "normal")
```

```
array(['normal', 'normal', 'attack', ..., 'normal', 'attack', 'normal'],
      dtype='<U6')
```

```
[126] data['attack'] = np.where(data['attack'] != 'normal', 'attack', 'normal')
```

```
[127] data['attack'].unique()
```

```
array(['normal', 'attack'], dtype=object)
```

nun müssen noch die Felder von Text in Zahlen umgewandelt werden da sie sonst nicht verrechnet werden können.

```
[128] from sklearn.preprocessing import LabelEncoder
```

```
[129] Der generierte Code unterliegt möglicherweise einer Lizenz | MihribanTekes/Detection-of-DDoS-Attacks-Using-Deep-Learning
le = LabelEncoder()
```

LabelEncoder ist ein Werkzeug zur Umwandlung kategorialer Daten in numerische Werte, damit sie von Machine-Learning-Modellen verarbeitet werden können (Die ML-Modelle können ja nur Zahlen verarbeiten, keine Worte)

```
0s [130] data['attack'] = le.fit_transform(data['attack'])
      data['flag'] = le.fit_transform(data['flag'])
      data['service'] = le.fit_transform(data['service'])
      data['protocol_type'] = le.fit_transform(data['protocol_type'])
```

```
0s [131] data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 125973 entries, 0 to 125972
Data columns (total 39 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   duration              125973 non-null  int64
 1   protocol_type         125973 non-null  int64
 2   service               125973 non-null  int64
 3   flag                 125973 non-null  int64
 4   src_bytes            125973 non-null  int64
 5   dst_bytes            125973 non-null  int64
 6   wrong_fragment       125973 non-null  int64
 7   hot                  125973 non-null  int64
 8   logged_in            125973 non-null  int64
 9   num_compromised      125973 non-null  int64
10   root_shell           125973 non-null  int64
11   su_attempted         125973 non-null  int64
12   num_root             125973 non-null  int64
13   num_file_creations   125973 non-null  int64
14   num_shells           125973 non-null  int64
15   num_access_files     125973 non-null  int64
16   is_host_login        125973 non-null  int64
17   is_guest_login       125973 non-null  int64
18   count               125973 non-null  int64
19   srv_count            125973 non-null  int64
20   serror_rate          125973 non-null  float64
21   srv_serror_rate      125973 non-null  float64
22   rerror_rate          125973 non-null  float64
23   srv_rerror_rate      125973 non-null  float64
24   same_srv_rate        125973 non-null  float64
25   diff_srv_rate        125973 non-null  float64
26   srv_diff_host_rate   125973 non-null  float64
27   dst_host_count       125973 non-null  int64
28   dst_host_srv_count   125973 non-null  int64
29   dst_host_same_srv_rate 125973 non-null  float64
30   dst_host_diff_srv_rate 125973 non-null  float64
31   dst_host_same_src_port_rate 125973 non-null  float64
32   dst_host_srv_diff_host_rate 125973 non-null  float64
33   dst_host_serror_rate 125973 non-null  float64
34   dst_host_srv_serror_rate 125973 non-null  float64
35   dst_host_rerror_rate 125973 non-null  float64
36   dst_host_srv_rerror_rate 125973 non-null  float64
37   attack               125973 non-null  int64
38   last_flag            125973 non-null  int64
dtypes: float64(15), int64(24)
memory usage: 37.5 MB
```

Die Python-Bibliotheken **NumPy** und **Pandas** sind zwei der wichtigsten Werkzeuge für die Datenverarbeitung und Analyse.

NumPy (Numerical Python):

- Ermöglicht schnelle mathematische Operationen mit Arrays und Matrizen.
- Bietet eine leistungsfähige Struktur für numerische Berechnungen, z. B. lineare Algebra, Statistik und Fourier-Transformationen.
- Besonders effizient durch die Nutzung homogener Arrays, die in C optimiert sind.

Pandas:

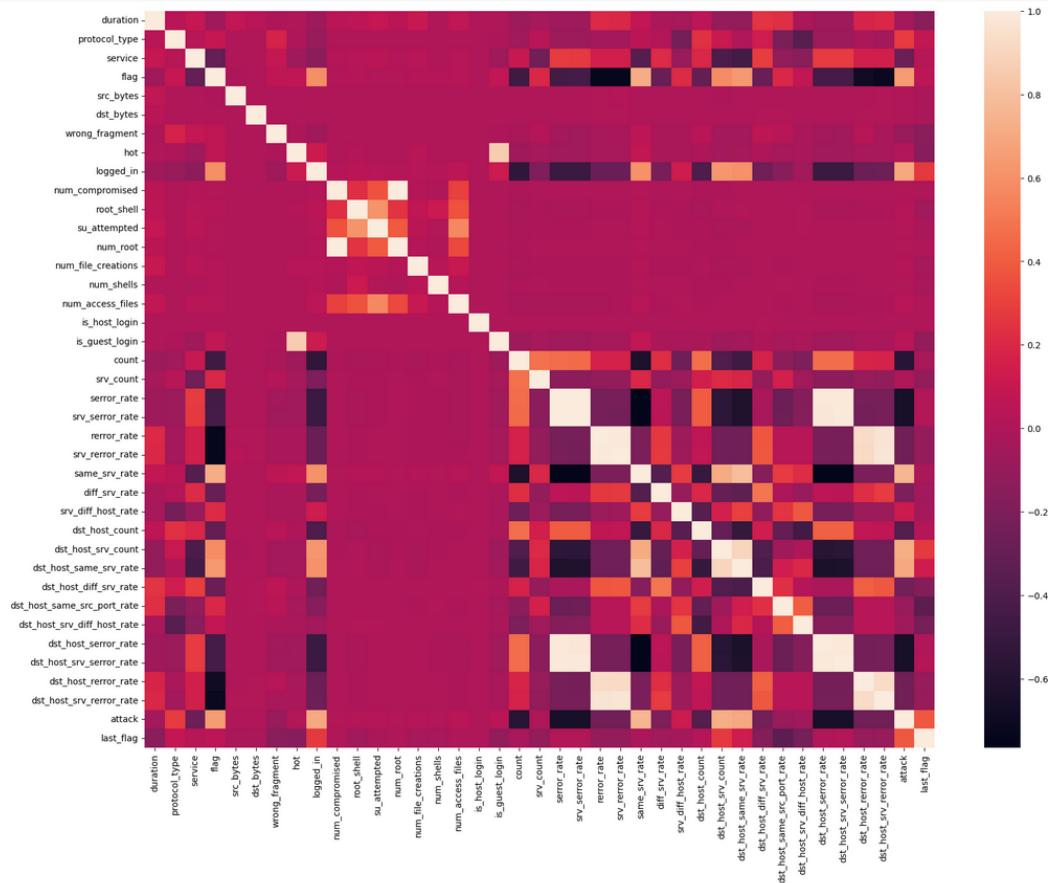
- Baut auf NumPy auf und bietet Datenstrukturen wie *DataFrame* und *Series* für die Tabellen- und Zeitreihenanalyse.
- Ermöglicht Datenmanipulation, Filtern, Gruppieren und Berechnungen auf grossen Datensätzen.
- Wird häufig für Datenaufbereitung vor dem Training von Machine-Learning-Modellen verwendet

Zur besseren Übersicht, welchen Einfluss die einzelnen Felder zueinander haben, kann man sich eine **Korrelations-Matrix** zeigen lassen. Eine Korrelationsmatrix ist eine Tabelle, die die Korrelationen zwischen mehreren Variablen zeigt. Jede Zelle in der Matrix enthält einen Korrelationskoeffizienten, der angibt, wie stark zwei Variablen miteinander zusammenhängen.

Die Werte reichen von -1 bis +1:
+1 bedeutet eine perfekte positive Korrelation (wenn der Wert einer Variablen steigt, so steigt auch der Wert der andere Variablen).
-1 bedeutet eine perfekte negative Korrelation (wenn der Wert einer Variablen steigt, so fällt der Wert der andere Variablen).
0 bedeutet, dass es keinen Zusammenhang zwischen den Variablen gibt.

```
[185] import matplotlib.pyplot as plt
plt.figure(figsize=(20, 15))

sns.heatmap(data.corr())
```



Bevor nun das Modell trainiert werden kann muss man einerseits die Werte skalieren sowie das Feld "attack" voneinander trennen. Wir möchten ein 'supervised learning' machen.

```
✓ [133] from sklearn.preprocessing import MinMaxScaler
```

```
✓ [134] scaler = MinMaxScaler()
```

```
✓ [135] X = data.drop("attack", axis=1)
```

```
✓ [136] X_scaled=scaler.fit_transform(X)
```

```
✓ [137] y = data["attack"]
```

```
✓ [138] from sklearn.model_selection import train_test_split
```

```
✓ [139] X_train, X_test, y_train, y_test = train_test_split(X_scaled, y)
```

```
✓ [140] X_train.shape
```

```
↔ (94479, 38)
```

Man skaliert die Werte vor der logistischen Regression, um sicherzustellen, dass alle Merkmale gleich gewichtet werden und das Modell effizient und korrekt "lernt".

Jetzt ist man soweit, den "Logistic Regression" Algorithmus zu verwenden

✓ Schritt 4: Trainieren des 'Logistic Regression' Algorithmus

```
✓ [141] Der generierte Code unterliegt möglicherweise einer Lizenz | dikers/aws-bigdata-analyze-demo  
0 s from sklearn.linear_model import LogisticRegression
```

```
✓ [142] lr = LogisticRegression()  
0 s
```

```
✓ [143] lr.fit(X_train, y_train)  
2 s
```

```
⇨  
▼ LogisticRegression ③ ②  
LogisticRegression()
```

```
✓ [144] preds = lr.predict(X_test)  
0 s
```

```
✓ [145] preds == y_test.values  
0 s
```

```
⇨ array([ True,  True,  True, ...,  True,  True,  True])
```

```
✓ [146] sum(preds == y_test.values) / len(preds)  
0 s
```

```
⇨ np.float64(0.9756461548231409)
```

Nun trainiert man das Modell.
Anschliessend vergleicht man
das Resultat mit einer "Test-
Datenmenge" um zu überprüfen,
wie genau das Ergebnis ist

wir haben also mit dem trainierten Modell eine Accuracy von ca. 97.55% erreicht, was sehr hoch ist.

▼ Schritt 5: finden des besten Sets von Parameter - "Hyperparameter optimization"

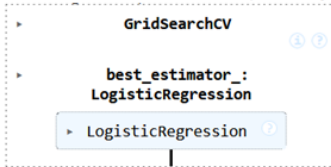
```
✓ [147] from sklearn.model_selection import GridSearchCV
```

```
✓ [148] param_grid = {  
    'C': [0.01, 0.1, 1, 10.100],  
    'penalty': ['l1', 'l2']  
}
```

```
✓ [149] lr = LogisticRegression()  
gs_cv = GridSearchCV(lr, param_grid, cv=5)
```

```
✓ [150] gs_cv.fit(X_train, y_train)
```

 /usr/local/lib/python3.11/dist-packages/sklearn/model_selection/_validation.py:528: FitFailedWarning:
20 fits failed out of a total of 40.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.



```
✓ [151] gs_cv.best_params_
```

```
 {'C': 10.1, 'penalty': 'l2'}
```

```
✓ [152] preds_new = gs_cv.best_estimator_.predict(X_test)
```

```
✓ [153] sum(preds_new == y_test.values) / len(preds_new)
```

```
 np.float64(0.9762494443386042)
```

Mit den optimierten Parametern sind wir nun auf einen Wert von 97.64% gekommen, also nicht viel besser.

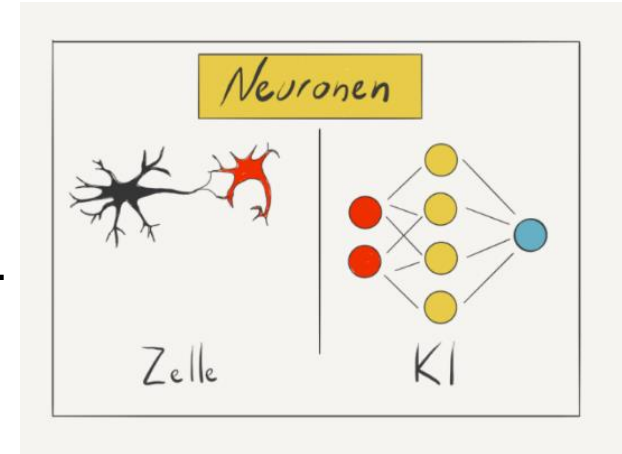
Hyperparameter-Optimierung ist der Prozess, bei dem man systematisch verschiedene Kombinationen dieser Parameter testet, um die beste Modellleistung zu erzielen – z. B. höchste Genauigkeit oder geringsten Fehler.

Anhang

- Einführung in Neuronale Netzwerke
- Quellen

Neuronale Netze

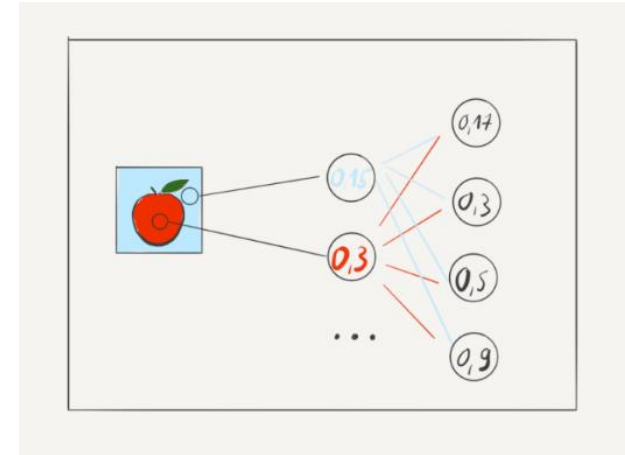
- Ausgeliehener Begriff aus der Biologie. Im menschlichen Gehirn ist ein Neuron eine Nervenzelle, die mit anderen Nervenzellen verbunden ist und elektrische Signale an diese weitergibt. Aus Milliarden von Neuronen setzt sich das Gehirn zusammen und aus ihrer Aktivität unser Geist.
- Wie die Nervenzelle verarbeitet auch ein künstliches Neuron Signale und gibt einen Output weiter
- Ein künstliches neuronales Netz funktioniert ähnlich:
Hier ist ein Neuron eine mathematische Formel, die einen Input verarbeitet und daraus einen Output generiert.
Die Werte der Formel werden dabei durch die Ausgangsdaten definiert. Viele künstliche Neuronen arbeiten zusammen und ergeben so ein künstliches neuronales Netz.



Quelle: <https://www.wfb-bremen.de/de/page/stories/digitalisierung-industrie40/was-ist-ein-neuronales-netz>

Neuronale Netze (2)

- Nimmt man z.B. ein neuronales Netz, welches Bilder von Äpfel von Bildern von Birnen unterscheiden soll. Damit neuronale Netze funktionieren, benötigen sie Daten, deren Ergebnis sie kennen, um aus diesen zu lernen. Dieser Prozess wird "Training" genannt – das Netz führt eine Berechnung aus und schaut, ob diese zum erwarteten Ergebnis passt. In diesem Beispiel braucht unser Netzwerk also Bilder von Äpfeln und Birnen, die mit einer Beschriftung versehen sind, welches von beiden sie sind.
- Was das neuronale Netz macht ist jetzt folgendes: Es nimmt ein Bild, löst die einzelnen Pixel in Daten auf (zum Beispiel einen Farbwert) und berechnet dann mit diesen Daten in einer komplexen Formel ein Ergebnis, das sie danach mit der Beschriftung vergleicht.
- Stimmen Ergebnis der Formel und Beschriftung überein, hat das Netzwerk ein Bild richtig erkannt.
- Stimmen Ergebnis der Formel und Ergebnis nicht überein, stimmt die Berechnung noch nicht und es muss noch weitertrainieren.

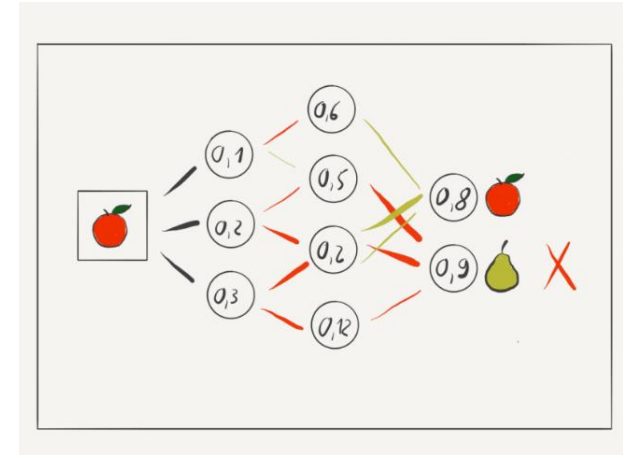


Neuronale Netze (3)

- Wie ist ein neuronales Netz nun aufgebaut?

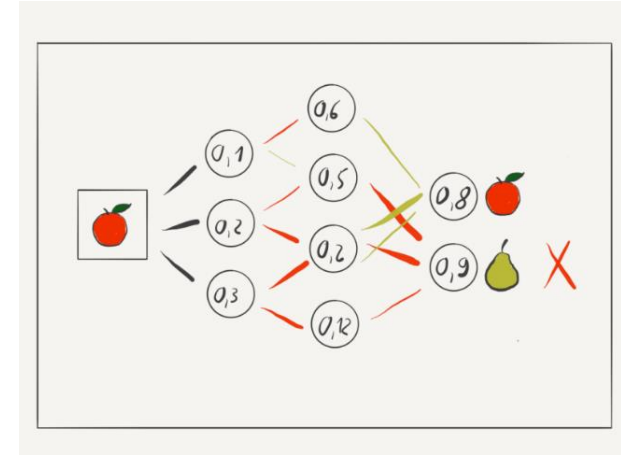
Es gibt sehr unterschiedliche Formen neuronaler Netze aber grundsätzlich besteht es aus mehreren Ebenen: einer Input-Ebene, die ein Signal aufnimmt, verschiedene Zwischenebenen, welche Berechnungen vornehmen und eine Output-Ebene, welches das Ergebnis der Berechnung bereitstellt. In diesem Beispiel ist ein neuronales Netz aufgebaut mit einer Inputebene, einer Zwischenebene und einer Output-Ebene

- Das Neuron der ersten Ebene nimmt nun den Input (den Farbwert des Pixels) und verarbeitet diese Information (berechnet eine Formel) und gibt danach das Ergebnis an die nächste Ebene mit Neuronen weiter.
- Die letzte Ebene mit Neuronen entspricht dann den beiden möglichen Outputs – Apfel oder Birne. Je nachdem, bei welchem der beiden Output-Neuronen das Ergebnis der Berechnung höher ist (in diesem Fall: näher am Wert 1 liegt), gibt das Netzwerk das Ergebnis als "Apfel" oder "Birne" heraus.



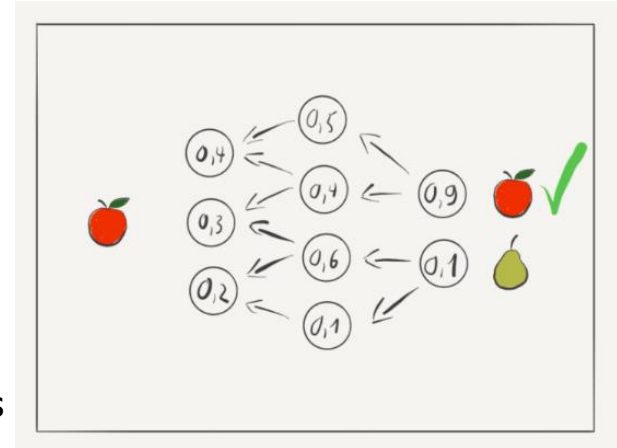
Neuronale Netze (4)

- Im Beispiel ist das Ergebnis dieser Berechnung aber falsch der Wert bei "Birne" ist höher als bei "Apfel", obwohl das Bild einen Apfel zeigt. Das ist aber okay so, denn wenn ein neuronales Netz zum ersten Mal mit Daten gefüttert wird, sind seine Ergebnisse immer zufällig und können daher auch zufällig falsch sein. Das liegt daran, dass die Parameter der mathematischen Formeln beim ersten Trainingsdurchgang zufällig gewählt werden müssen, denn Programmierer kennen die genaue Formel ja nicht, mit der es möglich ist, Äpfel und Birnen voneinander zu unterscheiden (sonst bräuchten sie ja keine KI).
- Jetzt kommt aber der wichtige Schritt: Die zufälligen Werte der einzelnen Neuronen werden nun ausgehend vom Ergebnis so angepasst, dass sie das richtige Ergebnis berechnen. Das macht das Netz automatisch durch eine weitere komplexe Formel. Es ist ein bisschen so wie im Matheunterricht: $2 + X = 4$. "2" ist der Input, "4" der Output - welchen Wert muss also "X" haben, damit das korrekte Ergebnis herauskommt?



Neuronale Netze (5)

- Ein neuronales Netz löst diese Berechnung nicht durch Wissen, sondern durch Ausprobieren – es optimiert die einzelnen Werte in den Neuronen solange, bis das tatsächliche Ergebnis dem gewünschten Ergebnis entspricht. Dabei werden üblicherweise tausende von Parametern gleichzeitig angepasst, in vielen sehr kleinen Schritten.
- Durch Abgleich des Ergebnisses mit dem Zielwert kann das NN seine Neuronen also allmählich trainieren, die richtige Antwort zu geben.
- Diese Schritte werden nun wiederholt – viele tausende bis Millionen Male und nicht nur mit einem Bild, sondern mit vielen verschiedenen Bildern von Äpfeln und Birnen. Jedes Mal verändern sich die Neuronenwerte dabei ein wenig. Sie werden ein klein wenig besser darin, Äpfel und Birnen zu unterscheiden, da die Anzahl der Beispiele, die sie richtig zugeordnet haben, stetig wächst. Dieser Prozess heisst "Lernen" und ist ungeheuer rechenaufwändig.

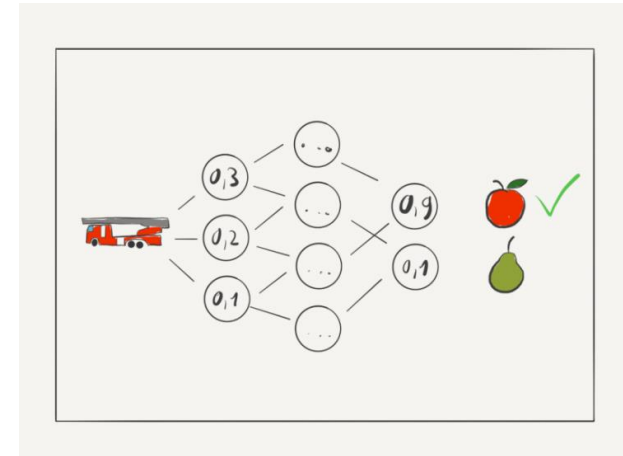


Neuronale Netze (6)

- Am Ende dieses Prozesses sind die Neuronen dann aber so fit, dass sie Bilder von Äpfeln von denen von Birnen unterscheiden können. Und hier kommt nun die "Magie" ins Spiel: Das neuronale Netz kann nicht nur die Bilder unterscheiden, die es bereits kennt und gelernt hat, richtig einzuordnen. Es kann dies auch bei Bildern, die es noch nie gesehen hat. Das Netz hat von den Trainingsbildern ein allgemeines Muster abstrahiert, das es nun auf neue Bilder anwenden kann.
- Die Unterscheidungsmerkmale von Äpfeln und Birnen hat das NN sich selbst im Laufe des Trainings beigebracht und sie entsprechen nicht menschlichen Massstäben. Wir würden diese Objekte unterscheiden nach Merkmalen wie etwa "ist rund und rot" -> Apfel, "ist länglich und grün" -> Birne. Ein Computer kann aber mit Begriffen wie "rot" oder "länglich" nichts anfangen. Er findet eigene Unterscheidungsmerkmale, die vielleicht heissen könnten "hat Durchschnittlich einen Rotwert von X Prozent" oder die sehr viel abstrakter sind. Sie stehen aber nirgendwo zum Nachlesen, sie sind implizit in der Gesamtformel enthalten, die alle Neuronen ausmachen.

Limitation von Neuronale Netzen

- Wird das Netzwerk mit Daten gefüttert, die nicht zu der Art der Trainingsdaten passen, ist es überfordert
- Das Äpfel-und-Birnen-Beispiel zeigt aber zugleich eine Limitierung von NNs auf: Werden sie plötzlich mit Bildern von Feuerwehrautos konfrontiert, können sie damit nichts anfangen – oder ordnen sie fälschlich als Äpfel ein.
- Um mit neuen, andersartigen Informationen umgehen zu können, müsste ein NN wiederum erneut trainiert werden. Es muss also einen Datensatz geben, der Antworten auf ein eng umfasstes Fragegebiet geben kann, damit ein neuronales Netz optimal arbeitet.



Weiterführende Quellen

- **Doktor Watson:** erklärt, wie ein neuronales Netz funktioniert, wie es lernt und was man damit macht - https://www.youtube.com/watch?v=ya_6l9IVMzY
- **Neural Network ausprobieren:** Auf <http://playground.tensorflow.org/> kann jeder einmal selbst mit einem neuronalen Netz im Browser herumspielen. Vor dem Start sollte man jedoch die Anleitung anschauen, denn sonst ist es schwer, die Bedeutung der verschiedenen Parameter zu verstehen.
- **Deep Learning 2023:** Zwei us-amerikanische KI-Profis erstellen jedes Jahr einen "State of AI"-Report, der die wichtigsten Entwicklungen in der Branche zusammenfasst. Hier die Präsentation von Ende 2023: <https://www.stateof.ai/>
- **CGP Grey:** Der Youtuber CGP Grey hat in einem kurzen Video die Funktion von einer speziellen Form von neuronalen Netzen, den Genetic Neural Networks, verständlich und anschaulich animiert zusammengefasst: <https://www.youtube.com/watch?v=R9OHn5ZF4Uo>

Quellen & weitere Infos

Datenquellen

- <https://github.com/jivoi/awesome-ml-for-cybersecurity#-tutorials>
- <http://www.secrepo.com/>

Beispiele

- <http://web.archive.org/web/20170514093208/http://fsecurify.com/using-machine-learning-detect-malicious-urls/>
- <http://web.archive.org/web/20170606022743/http://fsecurify.com/machine-learning-based-password-strength-checking/>

Email (Phishing)

- <https://monkey.org/~jose/phishing/>

Python

- <https://machinelearningmastery.com/ensemble-machine-learning-algorithms-python-scikit-learn/>

Tutorial

- <https://riptutorial.com/machine-learning>
- <http://web.stanford.edu/class/cs259d/>

Products

- <https://www.information-age.com/top-ai-cyber-security-companies-123487852/>

Data Preparation

- <https://searchbusinessanalytics.techtarget.com/definition/data-preparation>

Artikel

- <https://www.information-age.com/use-cases-for-ai-ml-cyber-security-123491392/>
- <https://towardsdatascience.com/machine-learning-for-cybersecurity-101-7822b802790b>
- <https://towardsdatascience.com/is-the-future-of-cyber-security-in-the-hands-of-artificial-intelligence-ai-1-2b4bd8384329>
- <https://www.udemy.com/course/data-science-and-data-preparation-with-knime/>
- <https://www.udemy.com/course/cybersecurity-data-science/>